PROJECT TITLE : TIMETABLE MANAGEMENT SYSTEM

BATCH – 5

VENUE – 35

IT – 3C

TEAM MEMBERS:

RA2311008020132 – PARIMALA DHARSHINI M

RA2311008020150 – VARSHITHA S

ABSTRACT

Developing a Timetable Management System using Python and MySQL involves integrating programming logic with database management to efficiently schedule and manage tasks or events. At its core, this system orchestrates the allocation of resources within specific times while accommodating various constraints and requirements. The Python application serves as the front-end interface, enabling smooth user interaction and data input, while MySQL functions as the back-end repository responsible for storing and retrieving timetable-related data. This abstraction captures the essence of the system's functionality and its significance in real-world scenarios. The Timetable Management System addresses the growing need for automated scheduling across diverse environments, including educational institutions, workplaces, and project-based teams, where optimizing time utilization is essential for boosting productivity and efficiency. Python's simplicity and versatility allow the application to implement scheduling algorithms, manage data structures, and provide user-friendly interfaces, ensuring both modularity and scalability. Integration with MySQL enables the system to store timetable data persistently, supporting efficient retrieval, updates, and manipulation through structured queries and database transactions. This enhances the reliability and robustness of the system, offering concurrent access, maintaining data integrity, and enabling data-driven insights for better decision-making. Finally, the abstraction highlights the system's user-centric design, focusing on intuitive interfaces and a seamless user experience. Whether through graphical interfaces or command-line tools, users can easily input scheduling data, view timetables, and manage tasks efficiently. The system's responsiveness and usability contribute to improved user satisfaction and encourage widespread adoption in daily operations.

TABLE OF CONTENTS

LIST OF FIGURE NAME / TABLE NAME

CHAPTER 1

INTRODUCTION

Creating a Timetable Management System using Python and MySQL combines the flexibility of programming with the efficiency of database management. Timetables are essential for organizing and managing tasks, schedules, and resources in various domains such as education, transportation, and business operations. This project aims to automate the process of timetable management, providing a systematic solution to handle complex scheduling scenarios. Python, a widely used and versatile programming language, serves as the foundation for developing the Timetable Management System. Its object-oriented features, ease of use, and extensive libraries make it an ideal choice for building scalable and efficient applications. By leveraging Python's simplicity and modular structure, developers can write maintainable and flexible code, allowing for easy future enhancements and updates. Integrating MySQL, a popular relational database management system, enhances the system's functionality by offering a structured and reliable repository for storing and retrieving timetable-related data. MySQL ensures data integrity and consistency through its ACID (Atomicity, Consistency, Isolation, Durability) properties, and provides scalability and reliability even under high-demand scenarios. Through the use of structured SQL queries, developers can efficiently manage data related to schedules, resources, and constraints. The collaboration between Python and MySQL equips the Timetable Management System with dynamic capabilities. Python manages user interaction and implements the core scheduling logic, while MySQL is responsible for data storage, retrieval, and manipulation. Using libraries such as MySQL-connector-python or SQL Alchemy, seamless communication is established between Python applications and MySQL databases, enabling real-time data exchange and synchronization.

CHAPTER 2

PROBLEM STATEMENT

A university requires an automated system to efficiently generate timetables for its courses, ensuring optimal allocation of available resources while accommodating the preferences of both students and faculty members. The system must enable administrators to input various constraints and preferences and automatically generate conflict-free, optimized timetables based on the provided data.

System Requirements

1. **User Management**
    - The system must support different access levels, including administrators, faculty members, and students.
    - Administrators should have the ability to manage users by adding, deleting, and updating user information.
2. **Course Management**
    - Administrators must be able to add, delete, and update course details, including course code, course name, credit hours, and prerequisites.
    - Each course should be assigned a designated faculty member responsible for teaching the course.
3. **Room Management**
    - Administrators should be able to manage room information, including adding, deleting, and updating details such as room number, seating capacity, and available facilities.

4. **Timetable Generation**
    - The system should enable administrators to define constraints and preferences, such as:
        - Preferred class timings,
        - Maximum consecutive classes allowed for a faculty member,
        - Room preferences and availability.
    - Based on the provided data, the system should automatically generate timetables for all courses, ensuring that there are no conflicts between faculty, rooms, and time slots.
    - Generated timetables must be displayed in a clear, user-friendly format, indicating the assigned faculty member, room, and time for each course.
5. **User Interface**
    - The system must offer an intuitive and easy-to-use interface for administrators to input data, manage users, courses, rooms, and view generated timetables.
    - Faculty members should have access to view their teaching schedules.
    - Students should be able to view the timetable for their enrolled courses.
6. **Database Integration**
    - The system must utilize a MySQL database to store all relevant data, including user accounts, course information, room details, and generated timetables.

- The system should allow efficient data retrieval, updates, and deletion through structured queries and robust database transactions.

7. **Security**
    - The system must implement proper authentication and authorization mechanisms to ensure that only authorized users can access, view, and modify data based on their roles.

Evaluation Criteria

- The completeness and correctness of the system's implementation.
- The user-friendliness and intuitiveness of the user interface for administrators, faculty members, and students.
- The system's efficiency and scalability in handling a large volume of data and concurrent users.
- Robustness in handling errors and preventing system crashes.
- Adherence to industry-standard software development practices, including:
    - Code readability and modularity,
    - Proper documentation,
    - Secure coding principles.

CHAPTER 3

SYSTEM REQUIREMENTS

3.1 Hardware Requirements

The hardware requirements for this system are minimal and are designed to be accessible for most modern computers. A basic system setup is sufficient to handle the development, execution, and database management tasks of the project.

- Computer: Any standard computer or laptop with a modern processor (Intel i3 or above / AMD equivalent) and at least 4 GB of RAM is capable of running Python, MySQL, and a code editor without performance issues. A more powerful system is recommended for handling large datasets or for multi-user testing.
- Storage: The system should have adequate free storage to install essential software such as Python, MySQL, and the chosen code editor. Additionally, sufficient space is needed for saving project files, datasets, backups, and generated timetables. A minimum of 2 GB free disk space is recommended for smooth operation.

3.2 Software Requirements

The software stack for the Timetable Management System consists of Python for programming and MySQL for database management, along with supporting libraries and tools to facilitate development.

- **Python:** Python 3.x is required as the main programming language. Its simplicity, versatility, and rich library ecosystem make it ideal for developing the logic, algorithms, and user interfaces for the timetable management system.

- **Code Editor or IDE:** Software like **Visual Studio Code** or **PyCharm** is recommended for writing, testing, and maintaining Python code. These tools offer syntax highlighting, debugging, and project organization features.
- **MySQL Database:** MySQL serves as the back-end relational database management system (RDBMS). It is used to store, retrieve, and manage user information, course details, room assignments, and the generated timetables. Tools like **MySQL Workbench** or **phpMyAdmin** can be used for easy visualization and management of the database.
- **MySQL Connector for Python:** The MySQL-connector-python library is essential for establishing a connection between the Python application and the MySQL database, enabling data insertion, retrieval, updates, and queries.
- **Additional Libraries:** Depending on the features of the system, libraries like Tkinter for GUI design, Pandas for data manipulation, or NumPy for numerical operations may also be required.

Together, these hardware and software components provide a solid foundation for building a dependable, flexible, and efficient Timetable Management System that can meet the scheduling needs of a university or similar institution.

CHAPTER 4

SYSTEM ARCHITECTURE

Architecture Diagram:

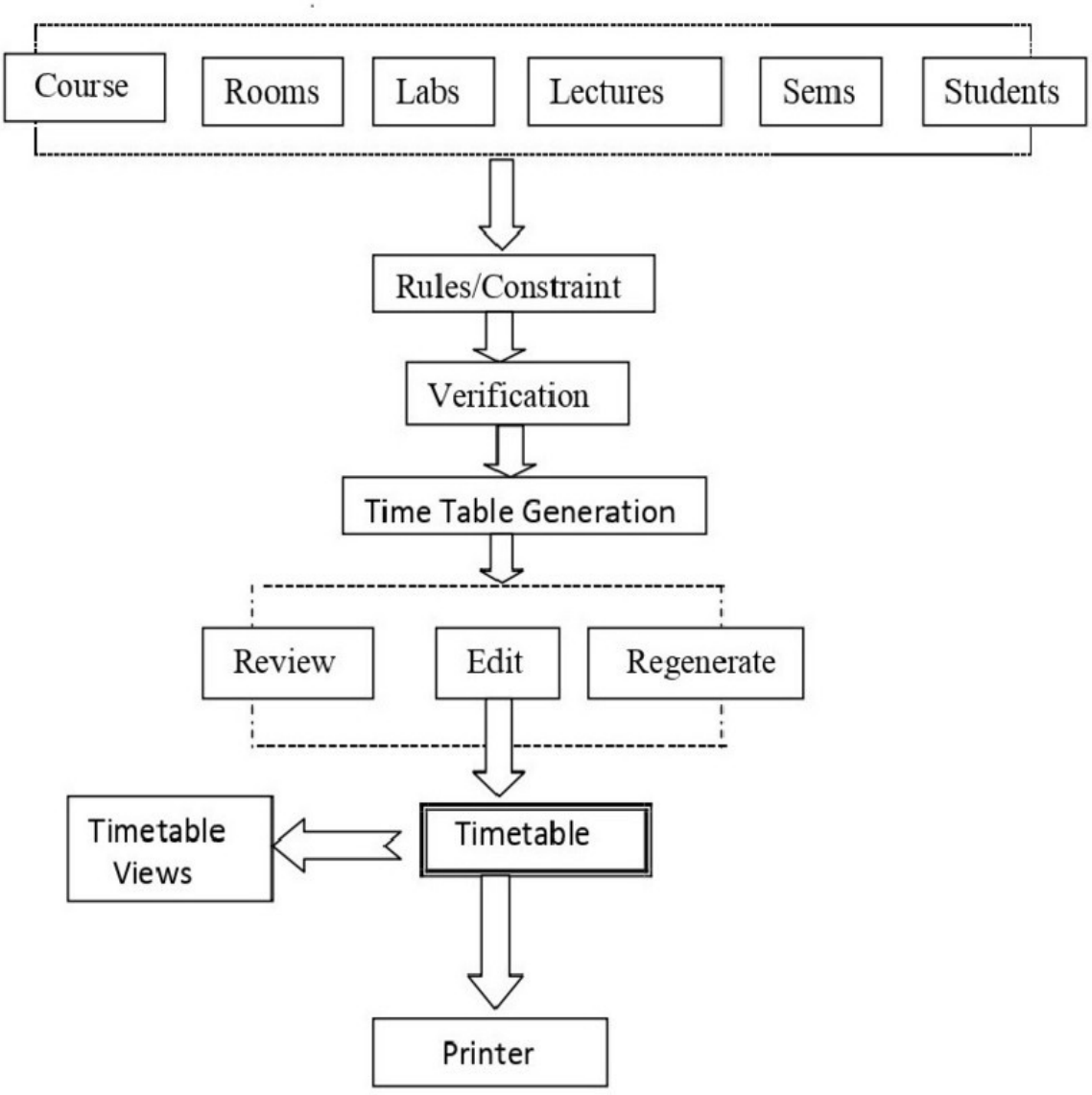


Fig – 4.1 SYSTEM ARCHITECTURE

The architecture of the **Timetable Management System** is designed to provide a structured and efficient approach to automate the scheduling of courses, rooms, and instructors while minimizing conflicts and accommodating preferences.

1. User Interface (UI):

- The User Interface serves as the primary access point for users such as administrators, faculty members, and students.
- Through intuitive input forms and user-friendly dashboards, users can add courses, update schedules, and view the generated timetable.
- It also allows customization of timetable views based on user roles and preferences to improve usability and accessibility.

2. Database:

- The database acts as the central repository for all timetable-related data. It stores course information, instructor availability, room details, time slots, and generated schedules.
- The database is designed to handle concurrent data access and maintain data integrity, ensuring that timetable conflicts and duplication of resources are systematically avoided.

3. Backend Services:

- Backend services form the core logic of the system. This layer processes user input, applies scheduling algorithms, and communicates directly with the database.
- These services handle the creation, updating, and deletion of schedule entries while resolving conflicts and adhering to constraints defined by the administrator.

- The backend ensures the timetable is optimized based on course requirements, room availability, and faculty preferences.

4. Authentication and Authorization:

- This component secures the system by controlling access to sensitive features and data.
- It uses authentication to verify user identity and applies role-based authorization to grant or restrict access to various system functionalities.
- Only authorized users such as administrators are allowed to modify timetable data, while students and faculty can only view their respective schedules.

5. Search and Filtering:

- Improve usability, the system includes search and filtering features that allow users to quickly locate specific courses, instructors, or room schedules.
- Users can search using keywords and apply filters based on time slots, room numbers, or faculty names, making navigation of the timetable smooth and efficient.

6. System Display:

- The display component is responsible for presenting the timetable and related information in a clear and organized format.
- Users can visualize complete schedules, receive alerts on conflicts, and get notifications about updates or changes to the timetable.
- This promotes easy access to up-to-date schedule information for all users.

CHAPTER - 5

SYSTEM IMPLEMENTATION

5.1 E-R DIAGRAM:

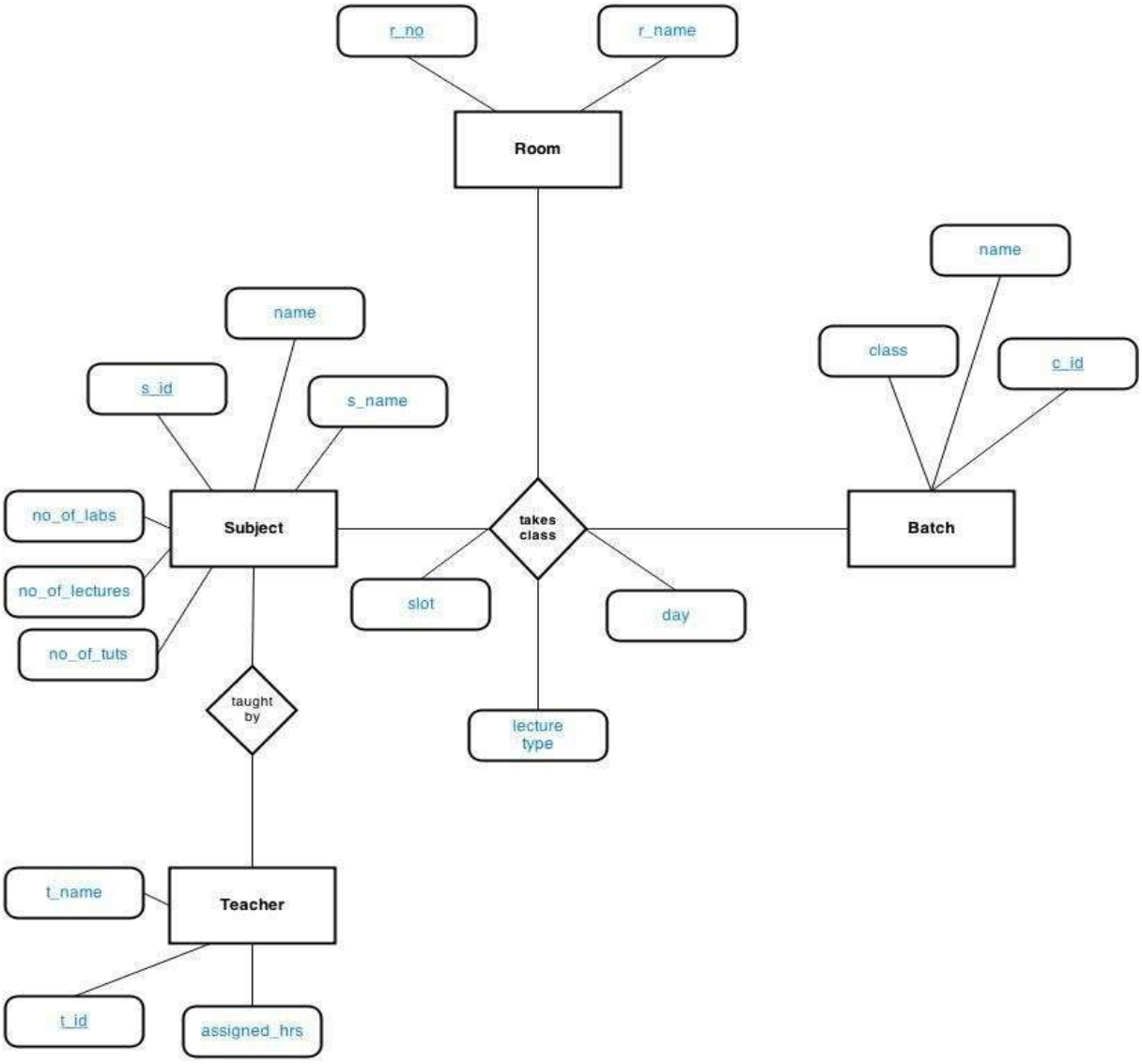


Fig – 5.1E-R DIAGRAM FOR TIMETABLE MANAGEMENT SYSTEM

5.2 SYSTEM MODULES:

The Timetable Management System is divided into various functional modules, each designed to handle a specific aspect of the system's operation. These modules work collaboratively to ensure smooth data flow, secure access, and effective timetable generation and management.

1. Database Connection Module:

- This module is responsible for establishing and managing a connection between the application and the MySQL database.
- It allows the system to store, retrieve, and update timetable-related data efficiently. Functions like connectToDatabase() and createTimetableTable() are used to initiate the connection and create necessary tables.

2. Timetable Management Module:

- This module handles core timetable operations such as adding new schedules, updating existing entries, and deleting obsolete data.
- Its main goal is to ensure that the timetable remains consistent and up to date while minimizing scheduling conflicts.

3. Display Timetable Module:

- This module retrieves the timetable data from the database and formats it for display, helping users to visualize schedules.
- It showcases important details like course names, assigned instructors, time slots, and room numbers in an organized manner.

4. Update Counts Module:

- This module provides statistical insights to the users by calculating the number of courses scheduled, instructor workloads, and the availability of free slots.
- It helps both administrators and faculty to get a summary view of the timetable's status.

5. User Authentication Module:

- Security is managed by this module, which ensures that only authorized users can access or modify data.
- Functions like verifyLogin() confirm that the correct username and password have been entered before allowing access to sensitive timetable operations.

6. GUI Design Module:

- This module focuses on creating an intuitive and user-friendly interface.
- Through input forms, buttons, and tables, users can easily interact with the system to add, update, delete, or view timetable data, improving the overall user experience.

7. Error Handling Module:

- This module ensures the system remains stable and user-friendly even during unexpected errors.
- It provides clear alerts and instructions when issues like invalid input or connection failures occur, allowing the user to correct problems smoothly.

CHAPTER 6

RESULTS AND DISCUSSION

The development of the Timetable Management System using Python and MySQL was conducted in progressive stages, each one enhancing functionality and improving the system's performance and usability. The table below highlights the comparison of features as the project evolved:

Development Stage	**Features Implemented**	**Outcome**
Stage 1: Database Setup	MySQL Database connection, table creation, and data insertion using Python.	Successfully established connection and stored course data securely.
Stage2: Core Functionality	Basic timetable creation and conflict-free schedule generation logic.	Generated simple timetables based on user input without time clashes.
Stage3: User Interface	Developed a user-friendly interface using Python Tkinter library.	Enabled users to input, update, and view timetables easily.
Stage4: Authentication Module	Integrated user login system for secure access to timetable data.	Provided restricted access based on user roles, enhancing security.
Stage5: Reporting & Error Handling	Implemented real-time error detection and statistical overview of scheduled data.	Improved system stability and gave users clear insights into scheduling status.

CHAPTER 7

CONCLUSION

The development of the Timetable Management System using Python and MySQL has been both a rewarding and educational journey. This project allowed us to explore and apply essential concepts of software development, database management, and user interface design in a real-world scenario. Through this system, we addressed the complexities associated with manually creating timetables for educational institutions, offering an automated, efficient, and conflict-free scheduling solution. Throughout the development process, Python proved to be an excellent programming language for its simplicity, versatility, and strong community support. Using Python's Tkinter library, we were able to create an intuitive Graphical User Interface (GUI) that made the system easy to navigate for users such as administrators, faculty, and students. The integration of MySQL as the backend database provided a reliable and secure environment for storing course, instructor, and timetable data, reinforcing our understanding of relational databases and structured query language (SQL).The system not only automates the generation of timetables but also ensures optimal resource utilization, minimizes manual errors, and saves considerable time for administrators. The user authentication module has added a layer of security, making sure that only authorized users have access to sensitive data and timetable configurations. In conclusion, the Timetable Management System stands as a practical example of how Python and MySQL can work together to solve real-world problems efficiently. The knowledge and skills acquired during this project will undoubtedly contribute to our growth as future developers, encouraging us to tackle more advanced and innovative software development challenges.

CHAPTER 8

FUTURE ENHANCEMENTS

The Timetable Management System developed using Python and MySQL lays a solid foundation for efficient and automated scheduling, but there is significant scope for future enhancements to improve its functionality, flexibility, and user experience. One potential enhancement is the integration of advanced scheduling algorithms such as genetic algorithms or constraint satisfaction techniques to further optimize timetable generation and automatically resolve scheduling conflicts. Additionally, the system could be expanded into a web-based application using frameworks like Django or Flask, allowing for remote access and collaboration from any device with internet connectivity. Another useful feature would be the implementation of real-time notifications and alerts via email or SMS to inform students and faculty of any last-minute changes to the timetable. Moreover, role-based dashboards could be introduced to offer tailored views for administrators, faculty, and students, making the system more interactive and user-friendly. Incorporating data analytics and reporting tools would allow educational institutions to assess resource utilization and scheduling trends over time, supporting better decision-making. Lastly, implementing cloud storage for the database and improving security with encrypted connections and multi-factor authentication would ensure data safety and scalability, making the system suitable for larger institutions with growing scheduling demands.

REFERENCES

https://dev.mysql.com/doc/

https://docs.python.org/3/

https://www.researchgate.net/publication/331242267_Automated_Timetable_Generation_A_Review

https://www.geeksforgeeks.org/timetable-scheduling-using-python/

https://docs.djangoproject.com/

https://dev.mysql.com/doc/connector-python/en/

APPENDIX 1

SAMPLE CODING

1. Database Module

```
-- Create database
CREATE DATABASE IF NOT EXISTS timetable_db;
USE timetable_db;
SET FOREIGN_KEY_CHECKS = 0;
-- Users table
CREATE TABLE IF NOT EXISTS users (
    user_id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50) UNIQUE NOT NULL,
    password VARCHAR(255) NOT NULL,
    name VARCHAR(100) NOT NULL,
    role ENUM('admin', 'faculty', 'student') NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;
-- Faculty table
CREATE TABLE IF NOT EXISTS faculty (
    faculty_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    designation VARCHAR(50) NOT NULL,
    department VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    phone VARCHAR(15),
    specialization VARCHAR(100),
    available_days SET('Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday') NOT NULL
) ENGINE=InnoDB;
```

```
-- Subjects table
CREATE TABLE IF NOT EXISTS subjects (
    subject_code VARCHAR(20) PRIMARY KEY,
    subject_name VARCHAR(100) NOT NULL,
    slot CHAR(1) NOT NULL,
    lecture_hours INT NOT NULL DEFAULT 3,
    tutorial_hours INT NOT NULL DEFAULT 0,
    practical_hours INT NOT NULL DEFAULT 0,
    credits INT NOT NULL,
    department VARCHAR(50) NOT NULL
) ENGINE=InnoDB;
-- Rooms table
CREATE TABLE IF NOT EXISTS rooms (
    room_id VARCHAR(20) PRIMARY KEY,
    building VARCHAR(50) NOT NULL,
    capacity INT NOT NULL,
    room_type ENUM('Lecture', 'Lab', 'Tutorial', 'Seminar') NOT NULL,
    equipment TEXT,
    is_air_conditioned BOOLEAN DEFAULT FALSE
) ENGINE=InnoDB;
-- Timetable slots table
CREATE TABLE IF NOT EXISTS timetable_slots (
    slot_id INT AUTO_INCREMENT PRIMARY KEY,
    day ENUM('Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday') NOT NULL,
    period INT NOT NULL,
    start_time TIME NOT NULL,
    end_time TIME NOT NULL,
    is_lunch BOOLEAN DEFAULT FALSE
   is_break BOOLEAN DEFAULT FALSE,
```

```
    UNIQUE KEY (day, period)
) ENGINE=InnoDB;
```

2. Login Module

```
{% extends "base.html" %}
{% block title %}Login{% endblock %}
{% block content %}
<!-- Full-height flex container to center content -->
<div class="d-flex justify-content-center align-items-center vh-0">
    <div class="col-md-6 col-lg-4">
        <div class="card shadow">
          <div class="card-header text-white text-center" style="background-color: #66A5AD;">
              <h4>Timetable Management System</h4>
           </div>
           <div class="card-body">
              {% if error %}
                 <div class="alert alert-danger">{{ error }}</div>
              {% endif %}
             <form method="POST" action="{{ url_for('login') }}">
                 <div class="mb-3">
    <label for="username" class="form-label">Username</label>
                      <input type="text" class="form-control" id="username" name="username"
required>
                 </div>
                 <div class="mb-3">
                    <label for="password" class="form-label">Password</label>
                  <input type="password" class="form-control" id="password" name="password"
required>
                 </div>
                 <button type="submit" class="btn btn-dark w-100">Login</button>
```

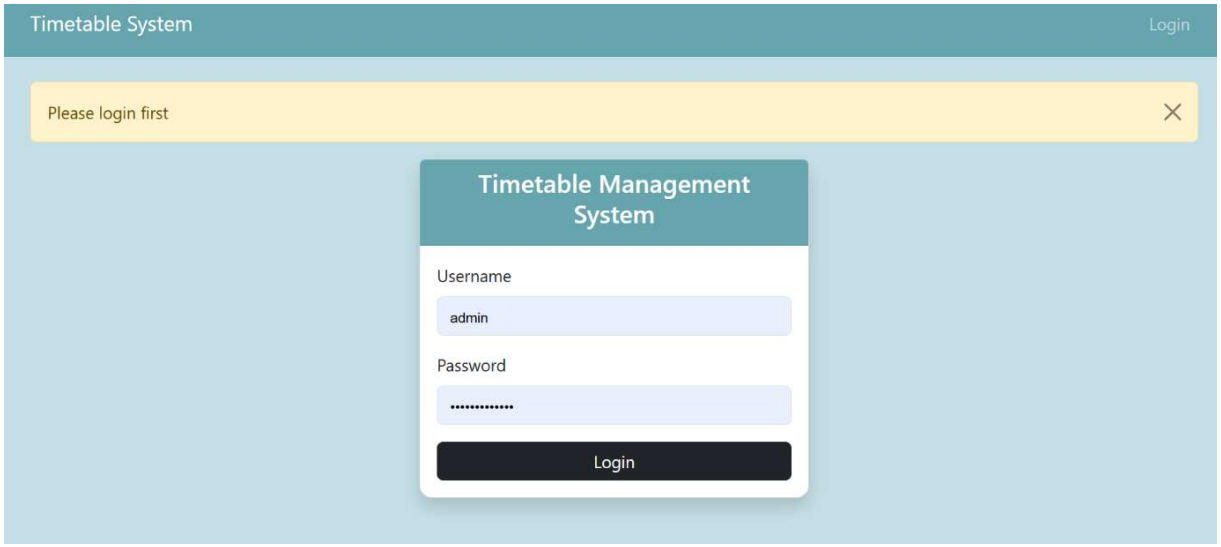
```
            </form>
          </div>
        </div>
    </div>
</div>
{% endblock %}
```

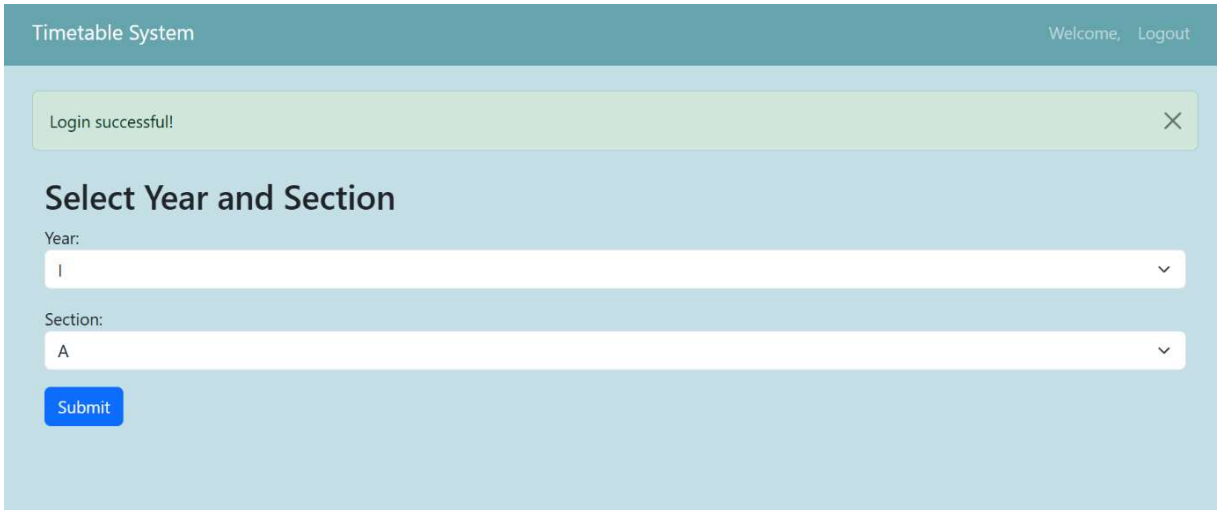
3. Main Module

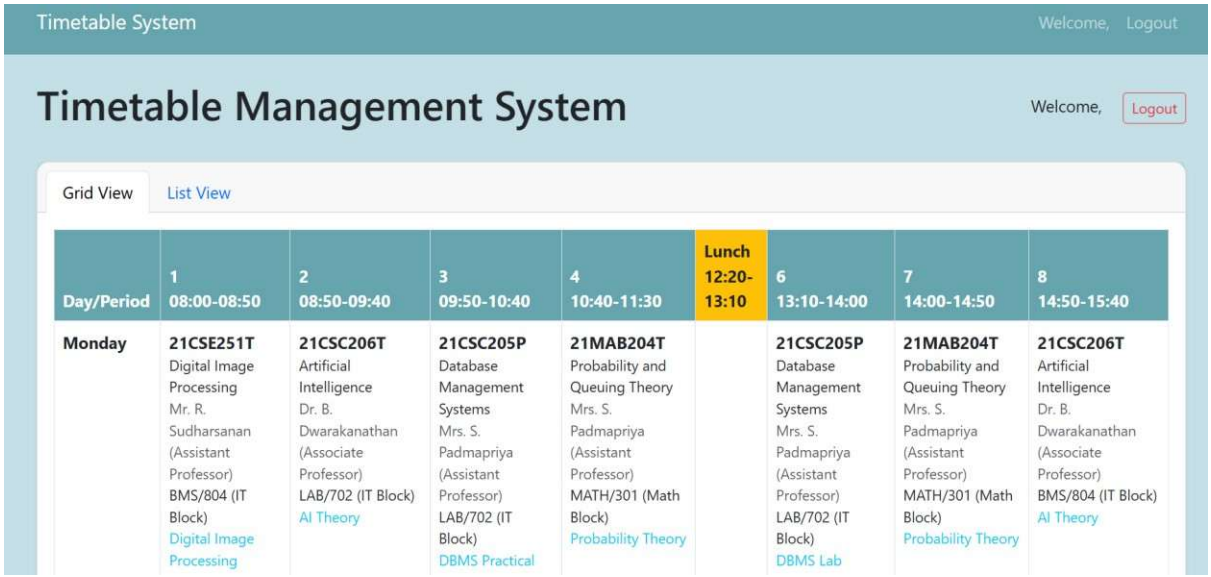
```
from flask import Flask, render_template, request, jsonify, session, redirect, url_for, flash
import mysql.connector
from mysql.connector import Error
from werkzeug.security import generate_password_hash, check_password_hash
from functools import wraps
import logging
import os
import subprocess
from datetime import timedelta
# Initialize Flask app
app = Flask(__name__)
app.secret_key = os.urandom(24).hex()
app.permanent_session_lifetime = timedelta(minutes=30)
# Configure logging
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(levelname)s - %(message)s',
    handlers=[
        logging.FileHandler('timetable.log'),
        logging.StreamHandler()
    ]
)logger = logging.getLogger(__name__)
```

APPENDIX 2

SAMPLE SCREENSHOT







Tuesday	**21CSC205P** Database Management Systems Mrs. S. Padmapriya (Assistant Professor) LAB/702 (IT Block) DBMS Practical	**21LEM202T** Universal Human Values Dr. Kulijeet Kaur (Assistant Professor) HUM/101 (Humanities Block) Human Values	**21CSC204J** Design and Analysis of Algorithms Dr. R. Kiruthiga (Professor) LAB/702 (IT Block) DAA Theory			**21CSC204J** Design and Analysis of Algorithms Dr. R. Kiruthiga (Professor) BMS/804 (IT Block) DAA Theory	**21CSC205P** Database Management Systems Mrs. S. Padmapriya (Assistant Professor) LAB/702 (IT Block) DBMS Lab	**21CSC206T** Artificial Intelligence Dr. B. Dwarakanathan (Associate Professor) LAB/702 (IT Block) AI Lab
Wednesday	**21CSC204J** Design and Analysis of Algorithms Dr. R. Kiruthiga (Professor) BMS/804 (IT Block) DAA Theory		**21CSE251T** Digital Image Processing Mr. R. Sudharsanan (Assistant Professor) BMS/804 (IT Block) Image Processing	**21MAB204T** Probability and Queuing Theory Mrs. S. Padmapriya (Assistant Professor) MATH/301 (Math Block) Probability Theory		**21CSC205P** Database Management Systems Mrs. S. Padmapriya (Assistant Professor) LAB/702 (IT Block) DBMS Lab	**21CSC205P** Database Management Systems Mrs. S. Padmapriya (Assistant Professor) LAB/702 (IT Block) DBMS Lab	

Subject Details

Code	Name	Slot	L	T	P	C	Faculty